

Received 16 March 2026, accepted 30 April 2026, date of publication 14 May 2026, date of current version 20 May 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3693469

RESEARCH ARTICLE

On the Optimality of Single-Label and Multi-Label Neural Network Decoders

YUNUS CAN GÜLTEKİN¹, (Member, IEEE), PÉTER SCHEEPERS², YUNCHENG YUAN³,
FEDERICO CORRADI¹, (Member, IEEE), AND ALEX ALVARADO¹, (Senior Member, IEEE)

¹Department of Electrical Engineering, Eindhoven University of Technology, 5612 AE Eindhoven, The Netherlands

²CatchSystems, 5708 HN Helmond, Netherlands

³University of Piraeus Research Center, 185 33 Piraeus, Greece

Corresponding author: Yunus Can Gültekin (y.c.g.gultekin@tue.nl)

This work was supported in part by Eindhoven Hendrik Casimir Institute (EHCI) Collaborative Projects Program.

ABSTRACT We investigate the design of two neural network (NN) architectures recently proposed as decoders for forward error correction: the so-called single-label NN (SLNN) and multi-label NN (MLNN) decoders. These decoders have been reported to achieve near-optimal codeword- and bit-wise performance, respectively. Results in the literature show near-optimality for a variety of short codes. In this paper, we analytically prove that certain SLNN and MLNN architectures can, in fact, always realize optimal decoding, regardless of the code. These optimal architectures and their binary weights are shown to be defined by the codebook, i.e., no training or network optimization is required. Our proposed architectures are in fact not NNs, but implement the maximum likelihood decoding rule differently. Optimal performance is numerically demonstrated for Hamming (7, 4), Polar (16, 8), and BCH (31, 21) codes. The results show that our optimal architectures (which are not NNs) are less complex than the SLNN and MLNN architectures proposed in the literature, which in fact only achieve near-optimal performance. Extension to longer codes is still hindered by the curse of dimensionality. Therefore, even though SLNN and MLNN can perform maximum likelihood decoding, such architectures cannot be used for medium and long codes.

INDEX TERMS Forward error correction, machine learning, maximum likelihood decoding, neural networks.

I. INTRODUCTION

Forward error correction (FEC) is essential for reliable data transmission over noisy channels [1]. FEC is also a fundamental part of next-generation wireless and optical communication systems [2], [3]. Designing FEC codes and decoders that achieve optimum performance, with low complexity and low latency, has been the objective of coding theory for decades. Maximum likelihood decoding minimizes the error probability, and thus, it is performance-wise optimal [4]. However, maximum likelihood decoding is impractical, as its complexity grows exponentially with increasing code blocklength n for a given coding rate $R = k/n$ [5].

In practice, instead of maximum likelihood decoding, low-complexity and low-latency suboptimal decoding algorithms are preferred. Bounded distance decoding (BDD) [6] or

decoding based on Chase-Pyndiah algorithms [7], [8] are widely used in combination with algebraic codes [9] for optical communications. Successive cancellation (SC) [10] and SC list [11] decoding of polar codes are used in 5G cellular communications [12]. Low-density parity-check (LDPC) codes, which also find application in quantum key distribution systems [13] and quantum error correction [14], are typically decoded via message-passing belief propagation (BP) [15], [16].

Following recent advances in machine learning, decoding FEC codes via neural network (NN) architectures has attracted considerable attention in the literature [17]. The advantages of these data-driven decoders are especially apparent for applications where no accurate mathematical model for the channel is available, or where the corresponding traditional decoder performs poorly for the code under consideration. As an example, consider quantum LDPC codes constructed via the Calderbank-Shor-Steane approach [18]. The Tanner graphs of these codes have short cycles of

The associate editor coordinating the review of this manuscript and approving it for publication was Andrea de Marcellis¹.

length 4. These cycles are known to significantly degrade the performance of BP and cause very high error floors [19]. As a solution to this problem, neural BP was recently proposed in [20]. Other examples of NN-based decoders include, e.g., [21], [22], [23].

Historically, NNs were first considered for decoding FEC codes in the early 1990s, e.g., in [24]. NN decoders were deemed suitable only for very short codes ($n < 100$) due to the curse of dimensionality [24], [25]. Even considering the recent surge in NN decoders proposed in the literature, starting with [26], we see from the comprehensive survey [17] and references therein that the codes considered for NN-based decoding are almost always limited to lengths below $n = 512$.

Recently, several decoder designs based on transformer architectures have been proposed. The latest examples of these decoders are the ones in [27], [28], [29], [30], [31], [32], [33], and [34]. While these works report promising performance, a careful examination reveals that their evaluations are often limited to comparisons against other transformer-based decoders. Moreover, in cases where classical decoders are included, widely adopted algorithms for specific code families are frequently omitted. For example, BP is used to decode BCH codes in [31], [32], and [34] instead of BDD or ordered-statistics decoding (OSD) [35]. Likewise, BP is employed for polar codes in [34] in place of SC or SC list decoding. Consequently, based on comparisons against classical decoders, we concluded in our earlier work [36] that transformer-based decoders generally remain less competitive than classical decoders, except in scenarios involving ultra-short blocklengths [33] or unconventional channel models [30].

More recently, single-label NN (SLNN) and multi-label NN (MLNN) decoders have been introduced [37], [38], [39], [40]. These decoders have a fully-connected architecture, with a few hidden layers. SLNN and MLNN decoders have been shown in [40] to provide codeword- and bit-wise near-optimal performance, respectively, for a variety of short ($n < 34$) FEC codes. The results in [40] were obtained by training the NNs at a fixed signal-to-noise ratio (SNR), and then, using the same NN for a wide range of SNRs. The NN architecture of these decoders does not rely on the knowledge of the specific code structure, i.e., they are model-free, and therefore, in principle, applicable to any code. Furthermore, these decoders are noniterative.

The computational complexity of the SLNN and MLNN decoders becomes infeasible for large input lengths k . To alleviate this problem, the iterative network pruning approach proposed in [41] has been used to prune weights in the SLNN and MLNN architectures in [42]. With this approach, the complexity of the SLNN and MLNN decoders has been reduced by up to 97 % without compromising the decoding performance [42, Sec. 6]. The reduction in complexity has been demonstrated for the Hamming (7, 4) and Polar (16, 8) codes. SLNN and MLNN with weight

pruning have the potential to be scalable for longer codes, without significantly sacrificing performance [42].

For the aforementioned reasons, at first glance, SLNN and MLNN decoders seem to be a very interesting alternative for decoding short codes. However, only heuristic rules for the design of these decoders exist in the literature. For example, it is unclear how many hidden layers or neurons per layer these decoders should use for different codes. Furthermore, to the best of our knowledge, the performance of SLNN and MLNN has not been rigorously analyzed in the literature, and no comparisons to traditional decoders exist.

In our earlier work [36], we studied the design and performance of SLNN and MLNN decoders. We proved analytically in [36, Theorem 1] and [36, Theorem 2], and that for certain non-fully-connected architectures with up to a single hidden layer, SLNN and MLNN decoders realize codeword- and bit-wise optimal decoding, respectively. The weights of these optimal architectures are binary and fully described by the codebook, i.e., no training is in fact required. Thus, these optimal architectures are not actually NNs but graphical representations of maximum likelihood decoding. Finally, we showed that already for blocklength $n = 31$, the NNs previously proposed in the literature are not competitive with respect to traditional (e.g., BDD) decoders. In this paper, we extend our study of SLNN and MLNN decoders in [36]. Our first contribution is to provide a more detailed discussion around the optimality analysis based on [36, Theorem 1] and [36, Theorem 2]. The second contribution is to extend our investigations to include a simulation study using polar codes which are among the most popular nowadays. Finally, our third contribution is a quantitative complexity analysis (see Table 1, ahead) to demonstrate that our optimal non-NN architectures are significantly less complex than the nonoptimal SLNN and MLNN decoders used in [37], [38], [39], and [40]. These optimal architectures might find applications in machine-type and ultra-low-latency communications to decode short blocklength codes [43], [44].

This paper is organized as follows. We provide preliminary information in Sec. II. SLNN and MLNN decoders are described and studied in Sec. III. Conclusions are given in Sec. IV.

II. PRELIMINARIES

We consider the communication system in Fig. 1. The transmitter is a concatenation of an FEC encoder and a binary phase-shift keying (BPSK) mapper. The k information bits $\mathbf{b} = (b_1, \dots, b_k)$ are encoded using an (n, k) linear block code into a codeword $\mathbf{c} = (c_1, \dots, c_n)$. In this paper, we consider three different codes: a Hamming code, a polar code, and a Bose–Chaudhuri–Hocquenghem (BCH) code. The codebook $\mathcal{C} = \{\mathbf{c}^{(1)}, \mathbf{c}^{(2)}, \dots, \mathbf{c}^{(2^k)}\}$ consists of all 2^k possible codewords. The code bits c_i are mapped to BPSK symbols $s_i \in \{-1, 1\}$, via $s_i = 2c_i - 1$, for $i = 1, \dots, n$. Since the BPSK mapping is a one-to-one function, we also consider the set $\mathcal{S} = \{\mathbf{s}^{(1)}, \dots, \mathbf{s}^{(2^k)}\}$ to be the codebook, where each

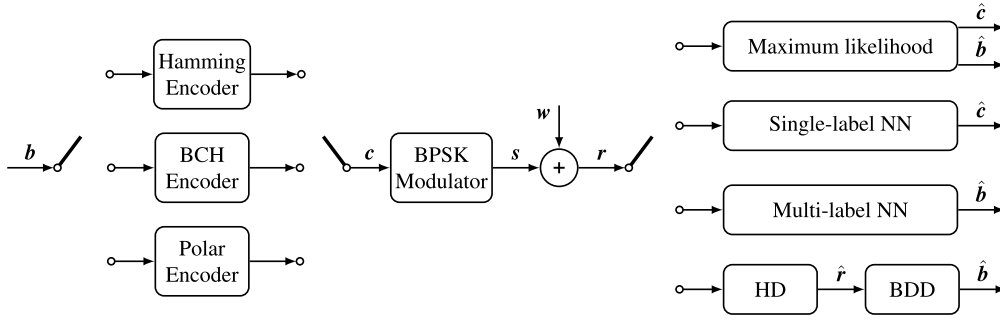


FIGURE 1. Communication system model considered in this work. Three possible encoders are used in combination with 4 different decoders. Maximum likelihood decoding is based on (2) or (8). The SLNN and MLNN decoders are those proposed in [37], [38], [39], [40]. BDD decoding is based on hard-decisions (HDs) $\hat{r}$ made on r .

element in this set is a length- n vector of symbols from the set $\{-1, +1\}$.

The symbols $\mathbf{s} = (s_1, \dots, s_n)$ are transmitted over an additive white Gaussian noise (AWGN) channel. At the receiver, real-valued noisy received symbols

$$\mathbf{r} = \mathbf{s} + \mathbf{w}, \quad (1)$$

are observed, where $\mathbf{w} = (w_1, \dots, w_n)$ is an n -dimensional vector of independent and identically distributed noise samples drawn from a zero-mean normal distribution with variance $\sigma^2 = N_0/2$, where N_0 is the power spectral density of the AWGN noise. Throughout this paper, we report results as a function of E_b/N_0 , where the energy per information bit E_b is $E_b = 1/(k/n)$, and where we assume equally likely symbols $s_i = \pm 1$.

As shown in Fig. 1, the receiver estimates the information bits using an FEC decoder. The decoder accepts as input either $\mathbf{r}$ (for soft-decision decoding), or hard detections $\hat{\mathbf{r}}$ made on $\mathbf{r}$ (for hard-decision decoding). In this paper, we consider optimal soft-decision maximum likelihood decoding, the SLNN and MLNN architectures from [37], [38], [39], and [40], and hard-decision BDD decoding. Maximum likelihood decoding and the basics required to understand SLNN and MLNN are described next.

A. MAXIMUM LIKELIHOOD DECODING

For any (n, k) linear block code whose codewords are transmitted with equal probability, the maximum likelihood decoding rule that minimizes frame (codeword) error rate (FER) estimates the transmitted codeword as

$$\hat{\mathbf{s}} = \underset{\mathbf{s} \in \mathcal{S}}{\operatorname{argmax}} p(\mathbf{r}|\mathbf{s}). \quad (2)$$

For a memoryless channel, (2) can be rewritten as

$$\hat{\mathbf{s}} = \underset{\mathbf{s} \in \mathcal{S}}{\operatorname{argmax}} \sum_{i=1}^n \log(p(r_i|s_i)). \quad (3)$$

For the AWGN channel in (1), the channel transition probability density function $p(r_i|s_i)$ is given by

$$p(r_i|s_i) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(r_i - s_i)^2}{2\sigma^2}\right). \quad (4)$$

Substituting (4) into (3) gives

$$\hat{\mathbf{s}} = \underset{\mathbf{s} \in \mathcal{S}}{\operatorname{argmax}} \left(-\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^n (r_i - s_i)^2 \right) \quad (5)$$

$$= \underset{\mathbf{s} \in \mathcal{S}}{\operatorname{argmax}} \sum_{i=1}^n 2r_i s_i - s_i^2 \quad (6)$$

$$= \underset{\mathbf{c}^{(j)} \in \mathcal{C}}{\operatorname{argmax}} \sum_{i=1}^n r_i c_i^{(j)}, \quad (7)$$

where (6) follows from neglecting the terms that do not depend on $\mathbf{s}$, and (7) from using $s_i = 2c_i - 1$, $s_i^2 = 1$, and again removing the terms that do not affect the maximization. A decoder that performs (7) is said to be codeword-wise optimal.

Similar to (2), the maximum likelihood decoding rule that minimizes bit error rate (BER) estimates the information bits as

$$\hat{b}_i = \underset{\beta \in \{0,1\}}{\operatorname{argmax}} p(\mathbf{r}|b_i = \beta), \quad (8)$$

for $i = 1, 2, \dots, k$, which is equivalent to

$$p(\mathbf{r}|b_i = 0) \stackrel{\hat{b}_i=1}{\leq} p(\mathbf{r}|b_i = 1) \quad (9)$$

$$\sum_{\mathbf{s}: b_i=0} p(\mathbf{r}|\mathbf{s}) \stackrel{\hat{b}_i=1}{\leq} \sum_{\mathbf{s}: b_i=1} p(\mathbf{r}|\mathbf{s}). \quad (10)$$

The left-hand side of (10) can be written as

$$\sum_{\mathbf{c}: b_i=0} \prod_{j=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(r_j - (2c_j - 1))^2}{2\sigma^2}\right), \quad (11)$$

which can be expanded, ignoring $1/\sqrt{2\pi\sigma^2}$, into

$$\sum_{\mathbf{c}: b_i=0} \exp\left(-\sum_{j=1}^n \frac{r_j^2 - 4r_j c_j + 2r_j + 4c_j^2 - 4c_j + 1}{2\sigma^2}\right). \quad (12)$$

Neglecting common terms (crossed out) in (12) yields

$$\sum_{c:b_i=0} \exp\left(\frac{2rc_T}{\sigma^2}\right) \stackrel{\hat{b}_i=1}{\leq} \sum_{\hat{b}_i=0} \exp\left(\frac{2rc_T}{\sigma^2}\right), \quad (13)$$

where $(\cdot)_T$ indicates the transpose operation. A decoder that performs (13) for $i = 1, 2, \dots, k$ is said to be bit-wise optimal.

B. NEURAL NETWORKS

In this work, the decoding of FEC codes via NNs is investigated. An NN is a series of linear transformations of real-valued vectors, from a $1 \times N_0$ input vector v_0 , to a $1 \times N_{L+1}$ output vector v_{L+1} . The transformations are defined via the recursive steps

$$v_i = \Gamma(v_{i-1}W_i + b_i), \quad (14)$$

for $i = 1, 2, \dots, L + 1$, where L is the number of hidden layers, and N_i is the number of neurons in the i^{th} layer. The input and output layers correspond to $i = 0$ and $i = L + 1$, respectively. Here, W_i is a $N_{i-1} \times N_i$ weight matrix, b_i is a $1 \times N_i$ bias vector, and

$$\Gamma(v_i) = [\sigma(v_{i1}), \sigma(v_{i2}), \dots, \sigma(v_{iN_i})], \quad (15)$$

is a nonlinear activation function. The most common activation functions are the ReLU function

$$\sigma(v_j) = \max(0, v_j), \quad (16)$$

the sigmoid function

$$\sigma(v_j) = 1/(1 + \exp(-v_j)), \quad (17)$$

and the scaled softmax function

$$\sigma(v_j) = \frac{\exp(\alpha v_j)}{\sum_{l=1}^{N_i} \exp(\alpha v_l)}, \quad (18)$$

where α is a scaling factor.

III. OPTIMALITY OF SLNN AND MLNN DECODERS

In this section, we will derive optimal SLNN and MLNN architectures. We will first show the results for the very simple Hamming (7, 4) code for illustrative purposes. At the end of this section, we will consider longer codes.

A. SINGLE-LABEL NEURAL NETWORK DECODERS

In [37] and [40], the decoding problem was formulated as a supervised single-label classification problem. The proposed SLNN decoder is a fully-connected NN with $N_0 = n$ input neurons, a hidden layer with N_1 neurons applying ReLU activation in (16), and $N_2 = 2^k$ output neurons using scaled softmax in (18) with $\alpha = 1$ to output a probability distribution over 2^k possible codewords. The estimated transmitted codeword corresponds to the neuron that outputs the highest probability, determined by the argmax function, as shown in Fig. 2.

Example 1 (SLNN for Hamming (7, 4)): The 7-7-16 SLNN architecture used to decode the Hamming (7, 4)

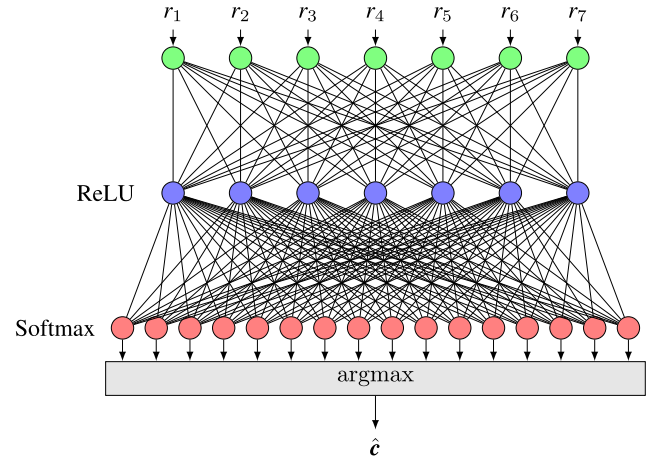


FIGURE 2. SLNN 7-7-16 architecture used in [40] to decode the Hamming (7, 4) code and obtain near-optimal performance.

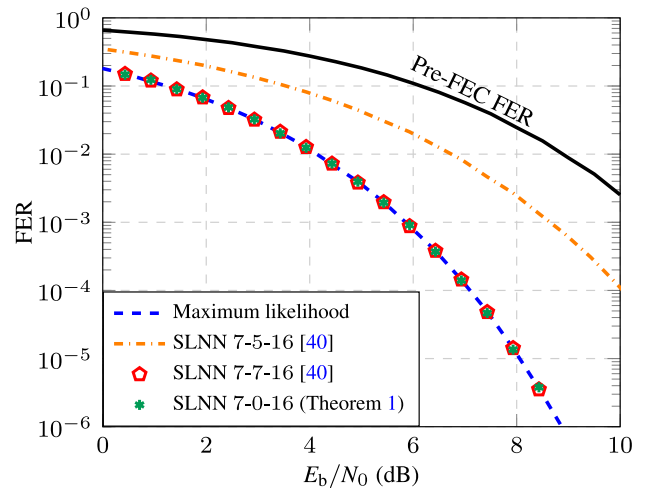


FIGURE 3. FER vs. E_b/N_0 for different SLNN decoders for the Hamming (7, 4) code. The results for SLNN 7-5-16 and SLNN 7-7-16 are based on our implementation of the architectures proposed in [40].

code and achieve codeword-wise near-optimal decoding performance in [40, Fig. 2 (a)] is shown in Fig. 2. The notation N_0 - N_1 - N_2 indicates the number of neurons in each layer. The corresponding FER results are shown in Fig. 3. The 7-7-16 SLNN decoder achieves codeword-wise optimal performance. On the other hand, the 7-5-16 SLNN decoder performs significantly worse. These results show that SLNN with a single hidden layer can approach optimal performance very closely, as long as the number of neurons in the hidden layer $N_1 \geq n$. This observation agrees with [40, Fig. 2 (a)].

We note that the SLNN decoders in Example 1 is trained based on a dataset that consists of pairs of one-hot encoded labels identifying the transmitted codewords and corresponding noisy received codewords. The dataset is generated via uniform random sampling from the codebook and transmission at $E_b/N_0 = 0$ dB, which has been shown to generalize well across SNRs and achieve near-optimal performance [40].

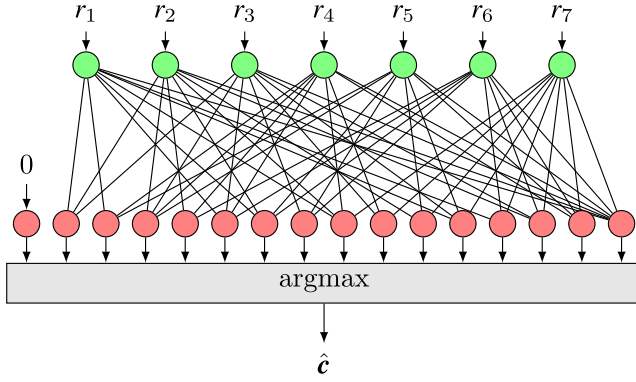


FIGURE 4. SLNN 7-0-16 architecture proposed in Theorem 1 to realize maximum likelihood decoding. The hidden layer in Fig. 3 is not present (it is not required for optimal performance). The number of edges here is 56, whereas for SLNN 7-7-16 in Fig. 3 the number of edges is 161.

The following theorem is the first contribution of this paper. It shows that the hidden layer in the SLNN architecture is, in fact, not needed. Optimal performance can always be guaranteed for any (n, k) linear block code with a NN with $N_0 = n$ input neurons and $N_2 = 2^k$ output neurons.

Theorem 1: Let (n, k) be a linear block code, with codebook $\mathcal{C}$ containing 2^k codewords. Consider a two-layer NN with n input neurons and 2^k output neurons. Let the n -by- 2^k weight matrix $\mathbf{W}_1$ connecting the input layer to the output layer be binary, and has its columns equal to the codewords in $\mathcal{C}$. This NN architecture is codeword-wise optimal. No training is required and bias vectors $\mathbf{b}$ are all zeros.

Proof: The summation in (7) can be expressed as a vector multiplication between $\mathbf{r}$ and $\mathbf{c}_T^{(j)}$, i.e.,

$$\hat{s} = \underset{s^{(j)} \in \mathcal{S}}{\operatorname{argmax}} \mathbf{r} \mathbf{c}_T^{(j)}. \quad (19)$$

The proof is completed by representing this maximization problem in matrix form: Consider a vector-matrix multiplication between the received signal $\mathbf{r}$ and the n -by- 2^k matrix $\mathbf{W}_1$ for which each column is equal to a different $\mathbf{c} \in \mathcal{C}$. The j^{th} element in the vector $\mathbf{r} \mathbf{W}_1$ is equal to the j^{th} metric $\mathbf{r} \mathbf{c}_T^{(j)}$ over which the maximization in (19) is carried out. This is what the SLNN architecture with $N_0 = n$, $N_1 = 0$, and $N_2 = 2^k$ realizes. ■

Theorem 1 shows that the optimal SLNN decoder, without any hidden layer, realizes codeword-wise maximum likelihood decoding. Furthermore, Theorem 1 also implies that no training is required, as the (binary) weight matrix of the NN consists of the codewords in the codebook. The number of edges in the resulting NN is the sum of the Hamming weights of all codewords in $\mathcal{C}$.

Example 1 (SLNN for Hamming (7, 4) continued): Fig. 4 shows the optimal SLNN 7-0-16 architecture that realizes optimal decoding of the Hamming (7, 4) code based on Theorem 1. We see that the neurons are sparsely connected. The leftmost neuron in the output layer is, in fact, not connected at all, which is due to the fact that it corresponds

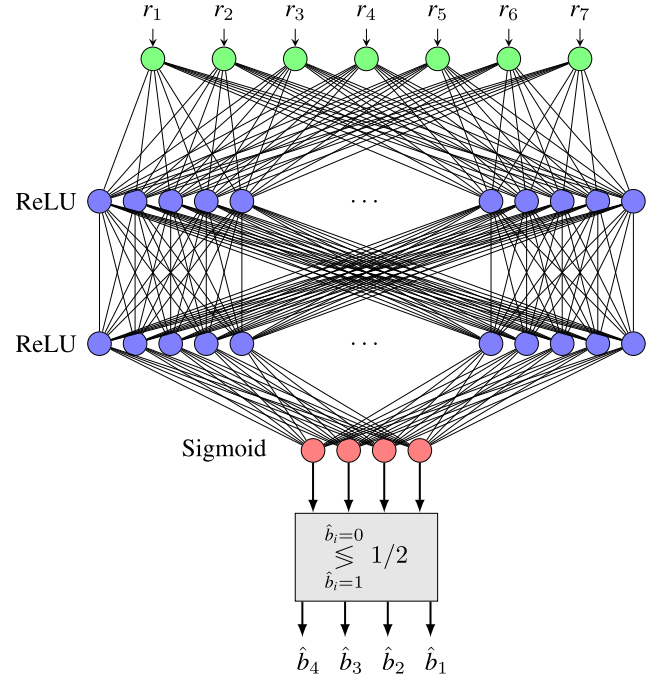


FIGURE 5. MLNN 7-50-50-4 architecture used in [40] to decode the Hamming (7, 4) code and obtain near-optimal performance.

to the all-zero codeword. Fig. 3 shows the corresponding FER performance. As Theorem 1 predicted, the presented results show that the hidden layer in SLNN 7-7-16 is indeed redundant: it only increases the computational complexity and memory requirements of the NN, and it also requires training. For this code, the number of edges in the NN is lowered from 161 (for SLNN 7-7-16) to only 56 (for SLNN 7-0-16).

B. MULTI-LABEL NEURAL NETWORK DECODERS

In [38], [39], and [40], the decoding problem was cast as a supervised multi-label classification problem. The proposed MLNN architecture is a fully-connected NN with $N_0 = n$ input neurons and $N_{L+1} = k$ output neurons applying sigmoid activation to output a probability distribution over each information bit. The estimated information bits are determined by a thresholding operation as in Fig. 5. The number of hidden layers L , the number of neurons N_i for $i = 1, 2, \dots, L$ in the hidden layers, and the activation functions of the hidden layers are design parameters.

Example 2 (MLNN for Hamming (7, 4)): The 7-50-50-4 MLNN architecture used to decode the Hamming (7, 4) code and achieve near-optimal bit-wise decoding performance in [40, Fig. 5 (a)] is shown in Fig. 5. This network uses ReLU activation in (16) in the hidden layers [40, Table 3]. The corresponding BER results are shown in Fig. 6. The 7-50-50-4 MLNN decoder approaches bit-wise optimal performance very closely. On the other hand, the MLNN 7-100-4 decoder, i.e., MLNN with a single hidden layer, performs significantly worse, as previously shown in [40, Fig. 5 (a)].

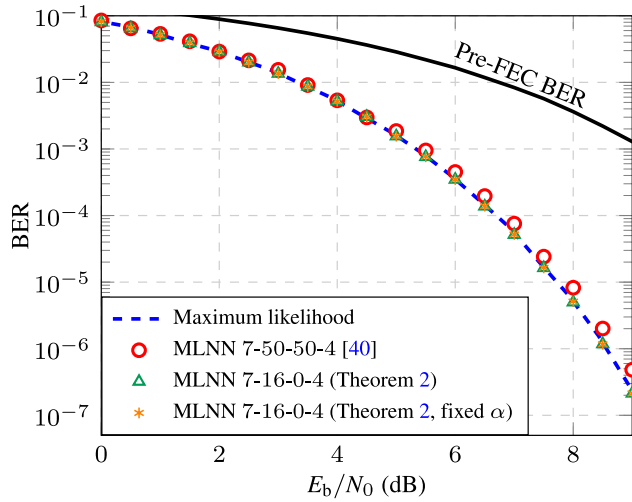


FIGURE 6. BER vs. E_b/N_0 for different MLNN decoders for the Hamming (7, 4) code. The results for MLNN 7-50-50-4 are based on our implementation of the architectures proposed in [40].

The MLNN decoder in Example 2 is trained based on a dataset that consists of pairs of information bit vectors that are encoded into the transmitted codewords and the corresponding noisy received codewords.

The following theorem is the second contribution of this paper. It shows that there exists an MLNN architecture with a single hidden layer that realizes bit-wise maximum likelihood decoding for any (n, k) linear block code with $N_0 = n$, $N_1 = 2^k$, and $N_2 = k$ input, hidden, and output neurons, respectively.

Theorem 2: Let (n, k) be a linear block code $\mathcal{C}$. Consider a three-layer NN with n input neurons, 2^k hidden neurons applying the scaled softmax in (18) with $\alpha = 2/\sigma^2$ as activation function, and k output neurons. Let the n -by- 2^k weight matrix $\mathbf{W}_1$ connecting the input layer to the hidden layer be binary, and has its columns equal to the codewords in $\mathcal{C}$. Let the 2^k -by- k weight matrix $\mathbf{W}_2$ connecting the hidden layer to the output layer be binary, and has its rows equal to all possible k -bit input strings of the code. We assume that the columns and rows of $\mathbf{W}_1$ and $\mathbf{W}_2$, respectively, are sorted such that j^{th} row of $\mathbf{W}_2$ is encoded into j^{th} column of $\mathbf{W}_1$ for $j = 1, 2, \dots, 2^k$. This NN architecture is bit-wise optimal. No training is required and bias vectors $\mathbf{b}$ are all zeros.

Proof: Dividing both sides of (13) by the sum of both sides, we get

$$\sum_{c: b_i=0} \frac{\exp\left(\frac{2rc_T}{\sigma^2}\right)}{\sum_c \exp\left(\frac{2rc_T}{\sigma^2}\right)} \hat{c}_i=1 \leq \sum_{c: b_i=1} \frac{\exp\left(\frac{2rc_T}{\sigma^2}\right)}{\sum_c \exp\left(\frac{2rc_T}{\sigma^2}\right)}. \quad (20)$$

In the NN defined in Theorem 2, the input-to-hidden transformation with the scaled softmax activation (18) computes the addends of the outer summations on both sides of (20). Then the hidden-to-output transformation computes the outer summation on the right-hand side. Observing also that the left and right sides of (20) add up to 1, a thresholding operation

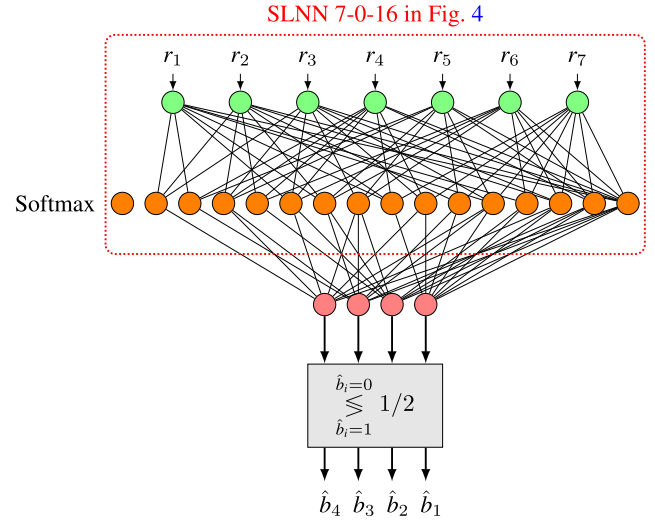


FIGURE 7. MLNN 7-16-4 architecture proposed in Theorem 2 to realize maximum likelihood decoding. The number of edges here is 88, unlike in Fig. 5, where 3200 edges are required.

applied to the output nodes of the NN, with threshold set at $1/2$, results in (13), and thus in (8). ■

Theorem 2 shows that the optimal MLNN decoder, with a single hidden layer, realizes bit-wise maximum likelihood decoding. This is not surprising since the universal approximation theorem implies that feedforward NNs having a nonpolynomial activation function with as few as one hidden layer are universal approximators. Furthermore, Theorem 2 also indicates that no training is required, as the (binary) weight matrices of the NN consist of the possible inputs and codewords of the code. The number of edges that connect the input layer to the hidden layer in the resulting NN is the sum of the Hamming weights of all codewords in $\mathcal{C}$. The number of edges that connect the hidden layer to the output layer in the resulting NN is the sum of the Hamming weights of all possible input strings, which is $k2^{k-1}$.

Example 2 (MLNN for Hamming (7, 4) Continued): Fig. 7 shows the optimal MLNN 7-16-4 architecture that realizes optimal decoding of the Hamming (7, 4) code based on Theorem 2. We see that the neurons are sparsely connected. The leftmost neuron in the output layer is not connected at all, similar to Fig. 4. Fig. 6 shows the corresponding BER performance. As Theorem 2 predicted, the presented results show that the second hidden layer and most neurons in the first hidden layer in MLNN 7-50-50-4 are redundant. For this code, the number of edges in the NN is reduced from 3200 (for MLNN 7-50-50-4) to only 88 (for MLNN 7-16-4). We note that the 100 hidden nodes of MLNN 7-50-50-4 in [40] apply ReLU activation. On the other hand, the 16 hidden nodes in our MLNN 7-16-4 apply softmax, which is computationally more complex, with an SNR-dependent scaling factor $\alpha = 2/\sigma^2$. However, it can be seen from Fig. 6 that performance is not sensitive to mismatch in α . When α computed for $E_b/N_0 = 4$ dB used for all SNRs, BERs are virtually the same.

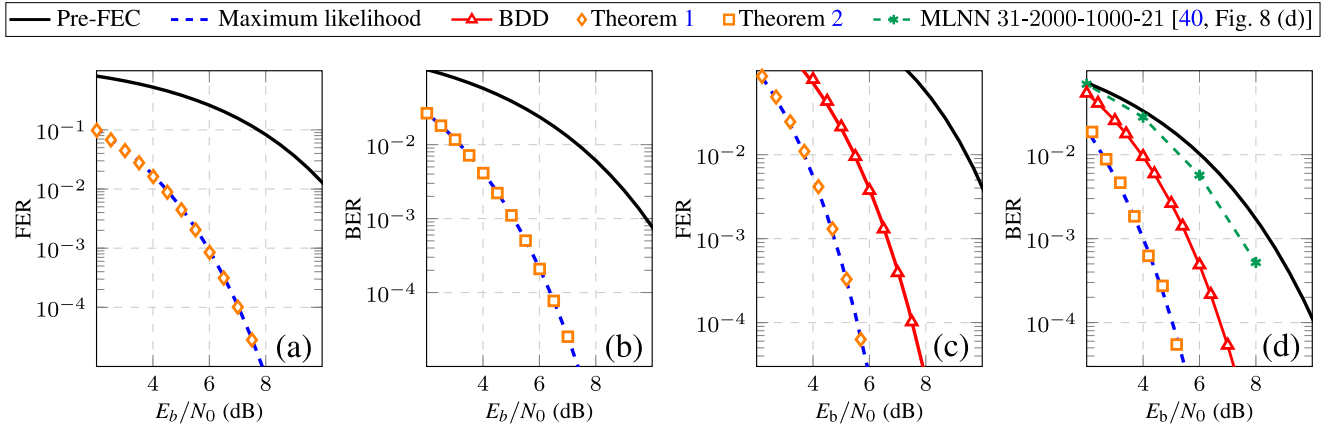


FIGURE 8. (a-b) FER and BER vs. E_b/N_0 of the Polar (16, 8) code for the SLNN and MLNN decoders. (c-d) FER and BER vs. E_b/N_0 of the BCH (31, 21) code for the SLNN, MLNN, and BDD decoders.

We note that the NN described in Theorem 1 corresponds to the first two layers of the NN described in Theorem 2 as shown within a dotted red rectangle in Fig. 7. Thus, the MLNN proposed in Theorem 2 can be seen as a “concatenation” of the SLNN proposed in Theorem 1 and an output layer. Finally, we emphasize that we are not using Theorems 1 and 2 to propose that the SLNN and MLNN decoders are the solution to the FEC decoding problem. We are merely demonstrating that what was proposed in the literature based on heuristic design rules is not optimal in terms of complexity or performance.

C. SCALABILITY TO LONGER CODES

As we discussed in Sec. I, scalability of NN decoders for longer FEC codes is hindered by the curse of dimensionality, i.e., the size of networks that provide adequate performance increases exponentially with k . This is indeed the case for the SLNN and MLNN decoders as they both have at least one layer where the number of neurons grows exponentially with k . Here, we study the performance of NNs defined in Theorems 1 and 2 for the (16, 8) polar code considered in [26] and for the (31, 21) BCH code considered in [40]. The properties of the corresponding NNs are given in Table 1. We see from Fig. 8 again that the NNs defined in Theorems 1 and 2 realize codeword- and bit-wise maximum likelihood decoding, respectively, for both codes.

The results for the (16, 8) polar code are shown in Fig. 8a and Fig. 8b. We note that the MLNN used in [26] was able to obtain bit-wise optimal performance. We also note that the MLNN used in [42, Fig. 4] was unable to achieve optimal performance for a (16, 8) polar code, although the objective there was to reduce complexity by pruning rather than to obtain the best performance. We see from Table 1 that both these MLNNs have a larger size than the optimal NNs described in Theorems 1 and 2, and require training. We believe that the MLNN in [26] being more complex than our optimum architecture can be attributed to the NN

TABLE 1. Properties of NN decoders considered in sec. III-C and in Fig. 8.

	Ref.	NN	Train	# Edges	Performance
Polar	[26]	$2^4 \cdot 2^7 \cdot 2^6 \cdot 2^5 \cdot 2^3$	Yes	12544, real	Bit-wise optimal
	[42]	$2^4 \cdot 2^8 \cdot 2^8 \cdot 2^4$	Yes	< 71689 ^a , real	Suboptimal
	◇	$2^4 \cdot 2^8$	No	2^{11} , binary	Codeword-wise optimal
	□	$2^4 \cdot 2^8 \cdot 2^3$	No	3072, binary	Bit-wise optimal
BCH	—*—	31-2000-1000-21	Yes	2.08M, real	Worse than BDD
	◇	31-2 ²¹	No	32.5M, binary	Codeword-wise optimal
	□	31-2 ²¹ -21	No	54.5M, binary	Bit-wise optimal

^a [42] starts from a $2^4 \cdot 2^8 \cdot 2^8 \cdot 2^4$ NN with 71689 edges, and achieves complexity reduction via weight pruning at the cost of a loss in performance.

having more layers and connections than necessary to achieve optimal performance.

The results for the (31, 21) BCH code are shown in Fig. 8c and Fig. 8d. In these figures, we also show the performance obtained by a very simple traditional hard-decision decoder: BDD (see Fig. 1). The performance of the MLNN decoder in [40, Fig. 8 (d)] is shown in Fig. 8d (green stars) and is worse than BDD. These results show that heuristically designed MLNN decoders are not competitive when compared to traditional decoders. This conclusion also holds for other NN-based decoders such as the error correction code transformer [27] and the cross-attention message passing transformer [28] as shown in [36, Sec. IV]. As expected, the results from Theorems 1 and 2 match those of maximum likelihood. However, as shown in the second to last column of Table 1, this comes at the cost of significant complexity. We note that optimal performance demanding significant complexity is in line with our expectations: maximum likelihood decoding is a difficult problem. Thus, we expect optimum decoding will only be realized for complexity-unconstrained applications or for ultra-short blocklengths, while practical suboptimal decoders such as SC and BDD will still be used when low-complexity solutions are required, e.g., in low-power edge devices.

IV. CONCLUSION

We provided a formal analysis of two neural network decoders for error-correcting codes: SLNN and MLNN decoders. These decoders have been shown in the literature to achieve codeword- and bit-wise *near-optimal* decoding performance, respectively. The designs in the literature are based on heuristic rules. In this paper, we showed analytically that these decoders, in fact, always realize optimal decoding for certain network architectures, without even requiring any training. This optimality was also numerically demonstrated for the Hamming (7, 4), polar (16, 8), and BCH (31, 21) codes. Moreover, these optimal architectures, which are not NNs but a way to realize maximum likelihood decoding, have binary weight matrices and are more sparsely connected than the ones achieving near-optimal performance. Our optimal decoders require at least one layer with size 2^k . Thus, our overall conclusion is that it is possible to achieve optimal decoding for codes with $k < 32$ with a “trivial” architecture that does not require training. Moreover, the complexity of these trivial architectures is smaller than that of “actual” suboptimal NNs that require training of real-valued weight matrices and bias vectors. On the other hand, for longer codes ($k > 32$), NNs that can achieve optimal performance quickly grow to impractical sizes, whereas smaller networks provide significantly poor performance.

ACKNOWLEDGMENT

An earlier version of this paper was presented in part at the 2025 International ITG Conference on Systems, Communications and Coding (SCC) [36].

REFERENCES

- [1] C. E. Shannon, “A mathematical theory of communication,” *Bell Syst. Tech. J.*, vol. 27, no. 3, pp. 379–423, Jul. 1948.
- [2] S. Miao, C. Kestel, L. Johannsen, M. Geiselhart, L. Schmalen, A. Balatsoukas-Stimming, G. Liva, N. Wehn, and S. T. Brink, “Trends in channel coding for 6G,” *Proc. IEEE*, vol. 112, no. 7, pp. 653–675, Jul. 2024.
- [3] E. Agrell, “Roadmap on optical communications,” *J. Opt.*, vol. 26, no. 9, Jul. 2024, Art. no. 093001.
- [4] A. Solomon, K. R. Duffy, and M. Médard, “Soft maximum likelihood decoding using GRAND,” in *Proc. IEEE Int. Conf. Commun. (ICC)*, Jun. 2020, pp. 1–6.
- [5] E. Berlekamp, R. McEliece, and H. van Tilborg, “On the inherent intractability of certain coding problems (Corresp.),” *IEEE Trans. Inf. Theory*, vol. IT-24, no. 3, pp. 384–386, May 1978.
- [6] E. Berlekamp, *Algebraic Coding Theory*. New York, NY, USA: McGraw-Hill, 1968.
- [7] D. Chase, “Class of algorithms for decoding block codes with channel measurement information,” *IEEE Trans. Inf. Theory*, vol. IT-18, no. 1, pp. 170–182, Jan. 1972.
- [8] R. M. Pyndiah, “Near-optimum decoding of product codes: Block turbo codes,” *IEEE Trans. Commun.*, vol. 46, no. 8, pp. 1003–1010, Aug. 1998.
- [9] A. Y. Sukmadji, U. Martínez-Peñas, and F. R. Kschischang, “Zipper codes,” *J. Lightw. Technol.*, vol. 40, no. 19, pp. 6397–6407, Oct. 15, 2022.
- [10] E. Arikan, “Channel polarization: A method for constructing capacity-achieving codes for symmetric binary-input memoryless channels,” *IEEE Trans. Inf. Theory*, vol. 55, no. 7, pp. 3051–3073, Jul. 2009.
- [11] I. Tal and A. Vardy, “List decoding of polar codes,” *IEEE Trans. Inf. Theory*, vol. 61, no. 5, pp. 2213–2226, May 2015.
- [12] *5G New Radio: Base Station Radio Transmission and Reception*, Standard 3GPP TS 38.104 V.15.2.0, Jul. 2018.
- [13] M. Milicevic, C. Feng, L. M. Zhang, and P. G. Gulak, “Quasi-cyclic multi-edge LDPC codes for long-distance quantum cryptography,” *npj Quantum Inf.*, vol. 4, no. 1, Apr. 2018.
- [14] P. Panteleev and G. Kalachev, “Quantum LDPC codes with almost linear minimum distance,” *IEEE Trans. Inf. Theory*, vol. 68, no. 1, pp. 213–229, Jan. 2022.
- [15] R. G. Gallager, “Low-density parity-check codes,” *IRE Trans. Inf. Theory*, vol. 8, no. 1, pp. 21–28, Jan. 1962.
- [16] F. R. Kschischang, B. J. Frey, and H.-A. Loeliger, “Factor graphs and the sum-product algorithm,” *IEEE Trans. Inf. Theory*, vol. 47, no. 2, pp. 498–519, Feb. 2001.
- [17] T. Matsumine and H. Ochiai, “Recent advances in deep learning for channel coding: A survey,” *IEEE Open J. Commun. Soc.*, vol. 5, pp. 6443–6481, 2024.
- [18] A. R. Calderbank, E. M. Rains, P. M. Shor, and N. J. Sloane, “Quantum error correction via codes over GF(4),” *IEEE Trans. Inf. Theory*, vol. 44, no. 4, pp. 1369–1387, Jul. 1998.
- [19] D. Poulin and Y. Chung, “On the iterative decoding of sparse quantum codes,” *Quantum Inf. Comput.*, vol. 8, no. 10, pp. 986–1000, Nov. 2008.
- [20] S. Miao, A. Schnerring, H. Li, and L. Schmalen, “Neural belief propagation decoding of quantum LDPC codes using overcomplete check matrices,” in *Proc. IEEE Inf. Theory Workshop (ITW)*, Apr. 2023, pp. 215–220.
- [21] Y.-H. Liu and D. Poulin, “Neural belief-propagation decoders for quantum error-correcting codes,” *Phys. Rev. Lett.*, vol. 122, no. 20, May 2019, Art. no. 200501.
- [22] A. Buchberger, C. Häger, H. D. Pfister, L. Schmalen, and A. Graell i Amat, “Pruning and quantizing neural belief propagation decoders,” *IEEE J. Sel. Areas Commun.*, vol. 39, no. 7, pp. 1957–1966, Jul. 2021.
- [23] A. Buchberger, C. Hager, H. D. Pfister, L. Schmalen, and A. Graell i Amat, “Learned decimation for neural belief propagation decoders : Invited paper,” in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Jun. 2021, pp. 8273–8277.
- [24] W. R. Caid and R. W. Means, “Neural network error correcting decoders for block and convolutional codes,” in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, San Diego, CA, USA, Dec. 2002, pp. 1028–1031.
- [25] X.-A. Wang and S. B. Wicker, “An artificial neural net Viterbi decoder,” *IEEE Trans. Commun.*, vol. 44, no. 2, pp. 165–171, Feb. 1996.
- [26] T. Gruber, S. Cammerer, J. Hoydis, and S. t. Brink, “On deep learning-based channel decoding,” in *Proc. 51st Annu. Conf. Inf. Sci. Syst. (CISS)*, Baltimore, MD, USA, Mar. 2017, pp. 1–6. [Online]. Available: <https://github.com/gruberto/DL-ChannelDecoding>.
- [27] Y. Choukroun and L. Wolf, “Error correction code transformer,” in *Proc. Adv. Neural Inf. Process. Syst.*, New Orleans, LA, USA, Dec. 2022, pp. 38695–38705.
- [28] S. Park, H.-Y. Kwak, S. Kim, Y. Kim, and J. No, “CrossMPT: Cross-attention message-passing transformer for error correcting codes,” in *Proc. Int. Conf. Learn. Rep. (ICLR)*, Singapore, Apr. 2024.
- [29] C. W. Lau, X. Shi, Z. Ziyang, C. Haiwen, and N. Guo, “Interplay between belief propagation and transformer: Differential-attention message passing transformer,” in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Ann Arbor, MI, USA, Jun. 2025.
- [30] J. Streitz, F. Weindel, and R. Heckel, “Transformer-based decoding in concatenated coding schemes under synchronization errors,” in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Ann Arbor, MI, USA, Jun. 2025, pp. 1–6.
- [31] S.-J. Park, H.-Y. Kwak, S.-H. Kim, S. Kim, Y. Kim, and J.-S. No, “Multiple-masks error correction code transformer for short block codes,” *IEEE J. Sel. Areas Commun.*, vol. 43, no. 7, pp. 2518–2529, Jul. 2025.
- [32] S. Xiao, W. Fang, and B. Chen, “Transformer-based decoders for cyclic codes: A tanner cycle-equivalent approach,” *IEEE Trans. Commun.*, vol. 73, no. 12, pp. 13173–13185, Dec. 2025.
- [33] H. Zhu, W. Xu, and X. You, “Latent-attention-based transformer for near ML polar decoding in short-code regime,” *IEEE Trans. Mach. Learn. Commun. Netw.*, vol. 4, pp. 373–390, 2026.
- [34] Y. Yan, J. Zhu, T. Zheng, Z. Xu, C. Jiang, and L. Dai, “Unified error correction code transformer with low complexity,” *IEEE Internet Things J.*, vol. 13, no. 8, pp. 15989–16000, Apr. 2026.
- [35] M. P. C. Fossorier and S. Lin, “Soft-decision decoding of linear block codes based on ordered statistics,” *IEEE Trans. Inf. Theory*, vol. 41, no. 5, pp. 1379–1396, Sep. 1995.

- [36] Y. Yuan, P. Scheepers, L. Tasiou, Y. C. Gültekin, F. Corradi, and A. Alvarado, "On the design and performance of machine learning based error correcting decoders," in *Proc. 14th Int. ITG Conf. Syst., Commun. Coding (SCC)*, Karlsruhe, Germany, Mar. 2025, pp. 1–6.
- [37] C. T. Leung, R. V. Bhat, and M. Motani, "Low-latency neural decoders for linear and non-linear block codes," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Waikoloa, HI, USA, Dec. 2019, pp. 1–6.
- [38] C. T. Leung, R. V. Bhat, and M. Motani, "Multi-label neural decoders for block codes," in *Proc. Int. Conf. Commun. (ICC)*, Jun. 2020, pp. 1–6.
- [39] C. T. Leung, M. Motani, and R. V. Bhat, "Multi-label and concatenated neural block decoders," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Jun. 2020, pp. 274–279.
- [40] C. T. Leung, R. V. Bhat, and M. Motani, "Low latency energy-efficient neural decoders for block codes," *IEEE Trans. Green Commun. Netw.*, vol. 7, no. 2, pp. 680–691, Nov. 2023.
- [41] J. Frankle and M. Carbin, "The lottery ticket hypothesis: Finding sparse, trainable neural networks," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, New Orleans, LA, USA, May 2018.
- [42] V. Malik, R. Ghosh, and M. Motani, "Achieving low complexity neural decoders via iterative pruning," in *Proc. Conf. Neural Inf. Process. Syst. (NeurIPS)*, Dec. 2021.
- [43] T. Tonnellier, M. Hashemipour, N. Doan, W. J. Gross, and A. Balatsoukas-Stimming, "Towards practical near-maximum-likelihood decoding of error-correcting codes: An overview," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Toronto, ON, Canada, Jun. 2021, pp. 8283–8287.
- [44] V. Papadopoulos, M. Hashemipour-Nazari, and A. Balatsoukas-Stimming, "Short codes with near-ML universal decoding: Are random codes good enough?" in *Proc. IEEE Workshop Signal Process. Syst. (SiPS)*, Coimbra, Portugal, Oct. 2021, pp. 94–98.

YUNUS CAN GÜLTEKİN (Member, IEEE) received the B.Sc. and M.Sc. degrees from Middle East Technical University, Ankara, Türkiye, in 2013 and 2015, respectively, and the Ph.D. degree from Eindhoven University of Technology (TU/e), The Netherlands, in 2020. From 2020 to 2023, he held a postdoctoral position at TU/e. Since January 2023, he has also been doing part-time freelance work. Since January 2024, he has been a permanent Research Staff Member at TU/e. His technical expertise and research interests lie in the intersection of information theory and communication theory, with a particular focus on the design of coded modulation and constellation shaping techniques for wireless, optical, and quantum communications. His research has received multiple awards, including Best Paper awards at the 2018 WIC/IEEE Symposium on Information Theory and Signal Processing in the Benelux and at the 2022 Optica Advanced Photonics Congress, the 2023 Quantum Delta NL Exchange Visit Grant, and the 2024–2025 International Excellence Fellowship from the Karlsruhe Institute of Technology. He is currently serving as an Associate Editor for IEEE COMMUNICATIONS LETTERS (Coding).

PÉTER SCHEEPERS received the B.Sc. and M.Sc. degrees in electrical engineering from Eindhoven University of Technology, Eindhoven, The Netherlands, in 2022 and 2024, respectively, with specialization in the connected world technologies track. He is currently employed as a Research Engineer with Industry. His research interests include information theory, digital signal processing, machine learning, and molecular communications.

YUNCHENG YUAN received the B.Sc. degree in measurement and control technology and instrument from Nanjing University of Posts and Telecommunications, in 2021, and the M.Sc. degree in artificial intelligence and engineering systems from Eindhoven University of Technology, in 2024. He is currently pursuing the Ph.D. degree with the University of Piraeus Research Center, Piraeus, Greece.

FEDERICO CORRADI (Member, IEEE) received the M.Sc. degree (cum laude) in physics from "La Sapienza" University of Rome and the joint Ph.D. degree in neuroinformatics from the University of Zurich and ETH Zürich, in 2015. He is currently an Assistant Professor with the Department of Electrical Engineering, Eindhoven University of Technology (TU/e), where he leads the Neuromorphic Edge Computing Systems (NECS) Lab. His research focuses on neuromorphic computing, ultra-low-power embedded systems, and bio-inspired algorithms for edge applications. Prior to TU/e, he held research positions at IMEC and IniLabs, contributing to the development of neuromorphic architectures for IoT and biomedical applications. He has co-authored over 50 peer-reviewed publications, holds multiple patents, and supervises students at all academic levels. He serves as an Associate Editor for IEEE TRANSACTIONS ON COGNITIVE AND DEVELOPMENTAL SYSTEMS and is a regular reviewer of journals, including *Nature* and *Frontiers*.

ALEX ALVARADO (Senior Member, IEEE) received the Electronics Engineering degree (Ingeniero Civil Electrónico) and the M.Sc. degree (Magíster en Ciencias de la Ingeniería Electrónica) from the Universidad Técnica Federico Santa María, Valparaíso, Chile, in 2003 and 2005, respectively, and the Licentiate of Engineering (Tecnologie Licentiatexamen) and Ph.D. degrees from the Chalmers University of Technology, Gothenburg, Sweden, in 2008 and 2011, respectively.

From 2011 to 2012, he was a Newton International Fellow with the University of Cambridge, U.K. From 2012 to 2014, he was a Marie Curie Intra-European Fellow with the University of Cambridge. From 2014 to 2016, he was a Senior Research Associate with the Optical Networks Group, University College London, U.K. He is currently a Full Professor with the Signal Processing Systems (SPS) Group, Department of Electrical Engineering, Eindhoven University of Technology (TU/e), The Netherlands. His research has been funded in part by Netherlands Organization for Scientific Research (NWO) via a VIDI Grant and by European Research Council (ERC) via an ERC Starting Grant. His general research interests include digital communications, coding, and information theory. His research has received multiple awards, including the Best Paper Award from the 2018 and 2023 Asia Communications and Photonics Conference, the 2019 OptoElectronics and Communications Conference, the 2022 Optica Advanced Photonics Congress, and the Best Poster Award from the 2009 IEEE Information Theory Workshop and the 2013 IEEE Communication Theory Workshop. He was a recipient of the 2015 IEEE Transactions on Communications Exemplary Reviewer Award and the 2015 Journal of Lightwave Technology Best Paper Award. He served as an Associate Editor for IEEE TRANSACTIONS ON COMMUNICATIONS (Optical Coded Modulation and Information Theory) from 2016 to 2018. From 2018 to 2020, he served on the OFC Subcommittee Digital and Electronic Subsystems (S4). He also served on the ECOC Subcommittee Theory of Optical Communications from 2019 to 2022.

...